

SportSlotReservation

Phase 2 Engineering Report — Backend API Foundation

Field	Value
Project	Multi-tenant SaaS for Indian residential community sports facility booking
Brand	Chandra AI Labs (chandraailabs.com)
Author	Chandra Nakkalakunta — Principal Cloud & AI/ML Architect
Period	2026-06-10 → 2026-06-12 (sub-phases 2.1–2.7, incl. corrective 2.6.x / 2.7.1)
Repository	github.com/chandranakkalakunta/sport-slot-reservation
GCP	Org chandraailabs.com · project sport-slot-dev · asia-south1 (Mumbai)
Status	PHASE 2 COMPLETE — deployed and validated over the public edge
Engineering method	Three-agent protocol: Strategist (Claude) · Worker (Claude Code) · human Coordinator

Prepared with Claude (Anthropic) acting as engineering Strategist under the project's three-agent protocol. All claims in this report were independently validated by the Coordinator in fresh terminal sessions.

1. Executive Summary

Phase 2 delivered the production-grade backend foundation: a FastAPI service with JWT authentication, five-layer tenant isolation, a tenant-scoped repository layer over Firestore, in-app rate limiting, a consistent error envelope with per-request tracing, containerization via Cloud Build, and deployment to Cloud Run in asia-south1 — validated end to end from local development to the public internet.

The phase closed with **11 documented issues**, each producing a durable engineering rule; total cloud spend of effectively **¥0** against a **¥5,000/month** budget; **89% test coverage** against an 80% gate; and zero credentials committed (one near-miss caught by independent validation). The most significant incident — ~30 hours of apparent deployment failure — resolved into two overlapping causes (a GCP reserved-path confound and a silent Google account review), documented fully in §6.

Metric	Result
Sub-phases completed	2.1–2.7 (+ corrective 2.6.1–2.6.3, 2.7.1)
Tests / coverage	31 passing · 89% (gate: 80%)
ADRs accepted	0006 API Design · 0007 Auth & Authorization · 0008 Data Layout & Repositories
Security charter	v1.2 — identity & credential model, org-policy exceptions, DPDP scope
Cloud spend	≈ ¥0 (free-trial credits intact: ~¥28,700)
Public validation	/health 200 · /readyz 200 (live Firestore) · 401/403 envelopes verified

2. Architecture

Request path: Client → Cloud Run edge → RequestId middleware → RateLimit middleware (slowapi subclass producing enveloped 429s) → Auth dependency (firebase-admin JWT verification + tenant/subdomain cross-check) → tenant-scoped repository → Firestore subcollections (/tenants/{tenant_id}/...).

Tenant isolation (ADR-0004), five layers: (1) permanent deny-all Firestore rules — clients can never touch the database directly; (2) repository pattern requiring a TenantContext at construction, making tenant scoping unbyypassable by code structure; (3) middleware cross-check of JWT tenant claims against the request host, with no admin bypass; (4) automated cross-tenant tests; (5) CI static-analysis gates (Phase 5).

Identity & credentials: admin@ is the sole human cloud identity; application credentials are always least-privilege service accounts — impersonated ADC locally (sa-firebase-admin), WIF for CI (sa-cloud-build), attached SA at runtime (sa-cloud-run). Zero JSON keys, enforced by org policy.

3. Key Decisions (ADRs 0006–0008)

Decision	Choice	Rationale
API versioning	URL path /api/v1/	Cacheability, debuggability
Error format	Envelope: code · message · request_id · timestamp; single code registry	Stable machine-readable codes; per-request tracing on every response
Pagination	Cursor-based only; offsets prohibited	Firestore offset(n) bills every skipped document — violates cost baseline
Health probes	Split liveness/readiness; liveness at /health (amended)	/healthz is a GCP reserved path intercepted at Google's frontend (§6)
JWT verification	firebase-admin verify_id_token() only; python-jose prohibited	Maintained SDK vs. unmaintained library with CVE history
Claims staleness	Accepted ≤1h window; check_revoked on sensitive ops	Per-request role lookups double Firestore reads for a Tier-2 threat
platform_admin	Valid only on dedicated admin host; tenant_id=null; no bypass branch	A skip-if-admin branch would defeat all five isolation layers

Decision	Choice	Rationale
Rate limiting	Phase 2 in-memory (slowapi) → Phase 3 Redis → Phase 7 Cloud Armor	Progression decided once to avoid re-litigation
Data layout	Per-tenant subcollections /tenants/{id}/...	Isolation becomes structural; cross-tenant queries require deliberate intent
Firestore rules	Permanently deny-all (all access via backend Admin SDK)	Rules guard against client SDKs and leaked configs, not the API path

4. Security Posture

Charter v1.2 principles: Defense-in-Depth · Least Privilege · Secure by Default · Zero Static Credentials · Privacy by Design (DPDP) · Fail Closed · Verify, Don't Trust. Phase 2 controls shipped: security envelope with request IDs, JWT validation with tenant cross-check, PII redaction in structured logs, Pydantic validation, rate limiting, deny-all Firestore rules deployed from version control via a guarded script.

Org-policy exception (documented): iam.allowedPolicyMemberDomains overridden to allowAll for project sport-slot-dev only — required for public run.invoker on a consumer API; app-layer JWT auth and deny-all rules stand behind it; org default remains strict; review scheduled at PROD setup (Phase 8).

5. Issue Log — 11 Issues, 11 Rules

#	Issue	Rule adopted
1	Out-of-protocol side-session code on a stray branch	All work flows through the protocol
2	Org-admin account active for daily work; assumed identity model was fictional	Security docs must describe reality, not aspiration
3	Pre-flight assumed files absent that existed (firebase.json, pyproject stub)	Pre-flight expectations come from verified state; summaries are maps, not territory
4	Toolchain check passed while a fresh need failed (PATH)	Shell sessions don't inherit config changes
5	slowapi 429 bypassed the error envelope (two failed fixes)	Verify third-party library behavior at the installed version before designing on it
6	Real API key edited into the tracked .env.example (near-miss)	Setup instructions specify HOW to create files; make dev-env added
7	Cloud Build ran as the default Compute SA	Every gcloud mutation ships in a script — including hotfixes
8	Three failed build fixes from memory (logging flag does not exist)	After two failed fixes, stop and read --help / documentation
9	allUsers binding rejected by Secure-by-Default org policy	Org-policy exceptions are scoped, documented, and reviewed at the PROD boundary
10	Post-interruption Worker fabricated documentation and self-validated it	Re-paste full prompts after interruptions; Worker self-validation cannot catch fabrication
11	30-hour '404 saga' — see §6	Verify platform reserved paths; vary one variable; complete the probe list

6. The Cloud Run 404 Investigation

After first deployment, every run.app URL returned Google's HTML 404. Roughly 30 hours of systematic elimination followed — service configuration, IAM, org policy, propagation, hostname, region, client network — all verified correct, with request logs empty throughout. Two overlapping causes were ultimately confirmed:

Cause 1 — reserved path. /healthz is a documented GCP reserved path (cloud.google.com/run/docs/issues): Google's frontend intercepts it on every *.run.app URL and returns 404 without reaching the container. Every cloud probe in the investigation used /healthz and only /healthz; /readyz was in the validation plan from the start but was never executed because the plan halted at the first failure. The 'decisive' differential test was confounded — the control service was probed at / while ours was probed at /healthz.

Cause 2 — silent account review. Audit logs proved that all three deployed services were removed overnight with zero DeleteService entries — removal below the audit plane, which only Google-internal systems perform — consistent with an account-review process on the days-old organization. After verification, redeployment succeeded and the first /readyz ever sent over the public edge returned 200.

Lessons: verify platform reserved paths at design time; a differential test must vary exactly one variable; complete the probe list even after the first failure — one data point cannot localize a fault. One additional probe on day one would have saved 30 hours.

7. Final Validation (over the public edge)

Probe	Result	Proves
GET /health	200 {"status":"ok"}	Container serving via Google's edge
GET /readyz	200 {"status":"ready"}	Attached SA → live Firestore read
GET /api/v1/users/me (no token)	401 AUTH_MISSING_TOKEN envelope	Auth required by default; envelope + request_id over the edge
GET /api/v1/users/me (tenant token)	403 TENANT_MISMATCH envelope	Isolation guard refuses unmapped hosts — correct until DNS (Phase 4)

Probe	Result	Proves
Local vertical slice (uvicorn)	200 — full profile JSON	JWT verify → tenant cross-check → scoped repository → Firestore document

8. Phase 3 Preview — Booking Engine

Decisions already taken in design discussion: Memystore Redis (Basic, 1GB, asia-south1) for distributed slot locks from day one, absorbed by trial credits (ADR-0009, with VPC connector); computed availability — bookings are the only stored slot artifacts, with deterministic booking IDs serving as lock keys; policy resolution (Tenant Override → Global Default) wired for the four booking-relevant parameters; quota checks inside the booking transaction (correctness over a marginal read). Sub-phases 3.1–3.6 run ADRs → policy/facilities → availability → transactional booking → cancellation → audit logging.